
Python and functional
programming

# Why Python is Functional?

- **First-class functions**: Functions can be passed, returned, and stored like variables.
- **Immutability preference**: Encourages avoiding side effects.
- **Comprehensions**: Elegant syntax for transforming data without explicit loops.
- **Lambda expressions**: Quick anonymous functions for inline operations.
- **Higher-order functions**: Built-in tools like map, filter, and reduce.
- **Functional libraries**: functools and itertools extend functional capabilities.
- **Mix of paradigms**: Python supports functional, object-oriented, and procedural styles.

# Functions as first-class objects

# Functions as first-class objects

- Functions are *first-class objects* in Python. What exactly does this mean?
- it basically means that whatever you can do with a variable, you can do with a function. These include:
  - Assigning a name to it.
  - Passing it as an argument to a function.
  - Returning it as the result of a function.
  - Storing it in data structures.
  - etc.

# Function Python

```
def func():
    print("We are in first function")
    def func1():
        print("This is first child function")
    def func2():
        print(" This is second child function")
    func1()
    func2()
func()
```

The function can be referenced and passed to a variable and returned from other functions as well.

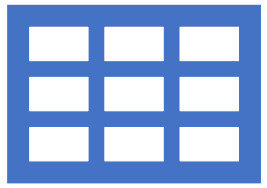
The functions **can be declared inside another function and passed as an argument to another function.**

A function that accepts other function as an argument is also called **higher order function**.

```
def add(x):
    return x+1
def sub(x):
    return x-1
def operator(func, x):
    temp = func(x)
    return temp
print(operator(sub,10))
print(operator(add,20))
```

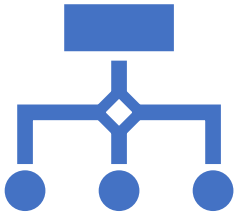
# List comprehensions

# List comprehensions



List comprehensions provide a nice way to construct lists where the items are the result of some operation.


The simplest form of a **list comprehension** is



Any number of additional for and/or if statements can follow the initial for statement. A simple example of creating a list of squares:

```
>>> squares = [x**2 for x in range(0,11)]
>>> squares
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

# List Comprehensions

- Here's a more complicated example which creates a list of tuples.

```
>>> squares = [(x, x**2, x**3) for x in range(0,9) if x % 2 == 0]
>>> squares
[(0, 0, 0), (2, 4, 8), (4, 16, 64), (6, 36, 216), (8, 64, 512)]
```

```
[(0, 0, 0), (2, 4, 8), (4, 16, 64), (6, 36, 216), (8, 64, 512)]
```

- The initial expression in the list comprehension can be anything, even another list comprehension.

```
>>> [[x*y for x in range(1,5)] for y in range(1,5)]
```

```
[[1, 2, 3, 4], [2, 4, 6, 8], [3, 6, 9, 12], [4, 8, 12, 16]]
```

# Lambda functions

# Functional Programming tools:Lambda functions

```
def f(x):
return x**2

print (f(8))
```

- Python allows for the use of a *lambda* function.
- Lambda functions are small, anonymous functions based on the lambda abstractions that appear in many functional languages.
- Right now, we'll take a look at some of the handy functional tools provided by Python.
- Lambda functions within Python.
  - Use the keyword *lambda* instead of *def*.
  - Can be used wherever function objects are used.
  - Restricted to one expression.
  - Typically used with functional programming tools.

```
g = lambda x: x**2

print(g(8))
```

## Using Lambda :

- Lambda definition does not include a “return” statement,
- it always contains an expression which is returned.
- We can also put a lambda definition anywhere a function is expected, and we don’t have to assign it to a variable at all.
- This is the simplicity of lambda functions.

## Without using Lambda :

- using def, we needed to define a function with a name and needed to pass a value to it.
- After execution, we also needed to return the result from where the function was called using the *return* keyword.

# Function factory

# Function Factory

- a.k.a. Closures.
- As first-class objects, you can wrap functions within functions.
- Outer functions have free variables that are bound to inner functions.
- A closure is a function object that remembers values in enclosing scopes regardless of whether those scopes are still present in memory.

```
def make_inc(x):
    def inc(y):
        # x is closed in
        # the definition of inc
        return x + y
    return inc

inc5 = make_inc(5)
inc10 = make_inc(10)

print(inc5(5)) # returns 10
print(inc10(5)) # returns 15
```

# Closure

- Closures are hard to define so follow these three rules for generating a closure:

1. We must have a nested function (function inside a function).
2. The nested function must refer to a value defined in the enclosing function.
3. The enclosing function must return the nested function.

```
def power_factory(exponent):

    def power(base):
        return base ** exponent              #uses parameter exponent

    return power  # return power

#using fanction factory

square = power_factory(2)  # Restituisce una funzione che eleva al quadrato
cube = power_factory(3)    # Restituisce una funzione che eleva al cubo

# use functions
print(square(5))  # Output: 25 (5^2)
print(cube(3))    # Output: 27 (3^3)
```

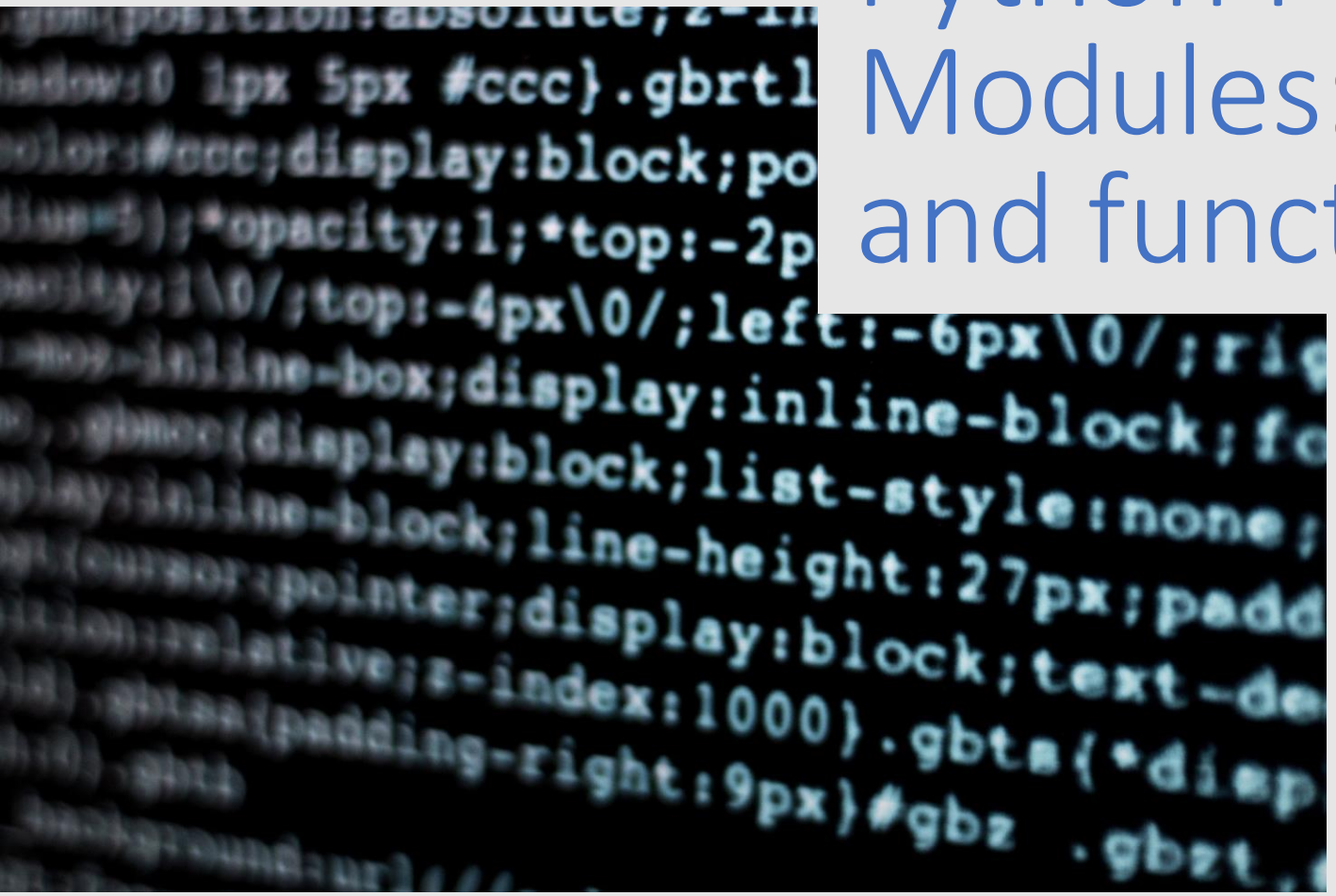
# Advantages of Factory Functions

- Modularity: You can create reusable functions with specific behaviors without duplicating code.
- Configurability: Factory functions allow you to customize the behavior of the returned functions based on the provided arguments.
- Use of Closures: The returned functions can access and use variables from the factory function, which is useful for maintaining state.

Common Application

- Creation of custom decorators.
- Generation of callbacks in graphical or networking libraries.
- Implementation of design patterns such as the Factory Method.

# Python Functional Modules: itertools and functools




ADVANCED TOOLS FOR EFFICIENT PYTHON PROGRAMMING

# itertools in Python

# Overview and Key Functions of itertools

| FUNCTION | DESCRIPTION | EXAMPLE |
|---|---|---|
| **count()** | Generates an infinite sequence of numbers | itertools.count(1, 2) |
| **cycle()** | Repeats elements of an iterable indefinitely | itertools.cycle(['A','B']) |
| **chain()** | Combines multiple iterables into one | itertools.chain([1,2],[3,4]) |
| **combinations()** | Generates r-length combinations | itertools.combinations('ABC',2) |
| **permutations()** | Generates r-length permutations | itertools.permutations('ABC',2) |

# functools in Python

# Key Functional Tools in Python

| FUNCTION | DESCRIPTION | EXAMPLE |
|---|---|---|
| **map()** | Applies a function to each item in an iterable | map(str.upper, ['a','b','c']) |
| **filter()** | Filters items in an iterable based on a condition | filter(lambda x: x > 0, [-1, 2, 0]) |
| **reduce()** | Applies a function cumulatively to an iterable | reduce(lambda x,y:x+y,[1,2,3]) |
| **partial()** | Creates a function with fixed arguments | partial(pow, 2) |
| **lru_cache()** | Caches function results for efficiency | @lru_cache(maxsize=128) |
| **wraps()** | Preserves metadata of decorated functions | @wraps(original_func) |

# Overview and Key Functions of functools

| FUNCTION | DESCRIPTION | EXAMPLE |
|---|---|---|
| **reduce()** | Applies a function cumulatively to an iterable | reduce(lambda x,y:x+y,[1,2,3]) |
| **partial()** | Creates a function with fixed arguments | partial(pow,2) |
| **lru_cache()** | Caches function results for efficiency | @lru_cache(maxsize=128) |
| **wraps()** | Preserves metadata of decorated functions | @wraps(original_func) |

# Functional programming tools: filter

**filter(*function, sequence*)**

- filters items from sequence for which function(*item*) is true.
- Returns a string or tuple if sequence is one of those types, otherwise result is a list

```
def even(x):
    if x%2==0:
        return True
    else:
        return False
for x in filter(even, range(0,30)):
    print(x)
```

```
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28]
```

# Using filter with lambda function

```
def even(x):
    if x%2==0:
        return True
    else:
        return False

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

even_numbers = filter(lambda x: x % 2 == 0, numbers)

even_numbers_list = list(even_numbers)

print(even_numbers_list)  # Output: [2, 4, 6, 8, 10]
```

# Features of filter()

- **Iterator**: filter() returns an iterator object. Therefore, if you want to view or use the results, you need to convert them to a list (or another type of sequence) using list(), tuple(), etc.
- **Lazy Evaluation**: Since filter() returns an iterator, the elements are filtered at the time of iteration, which can be more memory-efficient, especially with large sequences.
- **First-Class Functions**: filter() is an example of how Python supports functional programming, as you can pass functions as arguments and use anonymous functions like lambda.

# Functional programming tools: MAP

**map(*function, iterable*)**

applies function to each item in sequence and returns the results as a list.
Multiple arguments can be provided if the function supports it.

```
def expo(x, y):
    return x**y

for x in map(expo, range(0,5), range(0,5)):
    print(x)
```

[1, 1, 4, 27, 256]

```
def square(x):
    return x**2

for x in map(square, range(0,10)):
    print(x)
```

[0, 1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

# Map and lambDa function

- We can combine lambda abstractions with functional programming tools.
- This is especially useful when our function is small – we can avoid the overhead of creating a function definition for it by essentially defining it in-line.

```
numbers = [1, 2, 3, 4, 5]
squared_numbers = map(lambda x: x ** 2, numbers)
print(list(squared_numbers))
```

[1, 4, 9, 16, 25]

```
def add(x, y): return x + y

list1 = [1, 2, 3]
list2 = [4, 5, 6]
result = map(add, list1, list2)

result_list = list(result)

print(result_list)
```

```
#with lambda function
list1 = [1, 2, 3]
list2 = [4, 5, 6]
result = map(lambda x, y: x + y, list1, list2)
result_list = list(result)

print(result_list)
```

# reduce

**from functools import reduce**

**reduce(function, iterable[, initializer])**

returns a single value computed as the result of performing *function* on the first two items, then on the result with the next item, etc.

There's an optional third argument which is the starting value.

```
from functools import reduce

# Lista di numeri
numbers = [1, 2, 3, 4, 5]

def add(x, y):
    return x + y

result = reduce(add, numbers)

print(result)  # Output: 15
```

# reduce

```
import functools

# initializing list
lis = [ 1 , 3, 5, 6, 2, ]

# using reduce to compute sum of list
print ("The sum of the list elements is : ",end="")
print (functools.reduce(lambda a,b : a+b,lis))

# using reduce to compute maximum element from list
print ("The maximum element of the list is : ",end="")
print (functools.reduce(lambda a,b : a if a > b else b,lis))
```

# Features of reduce()

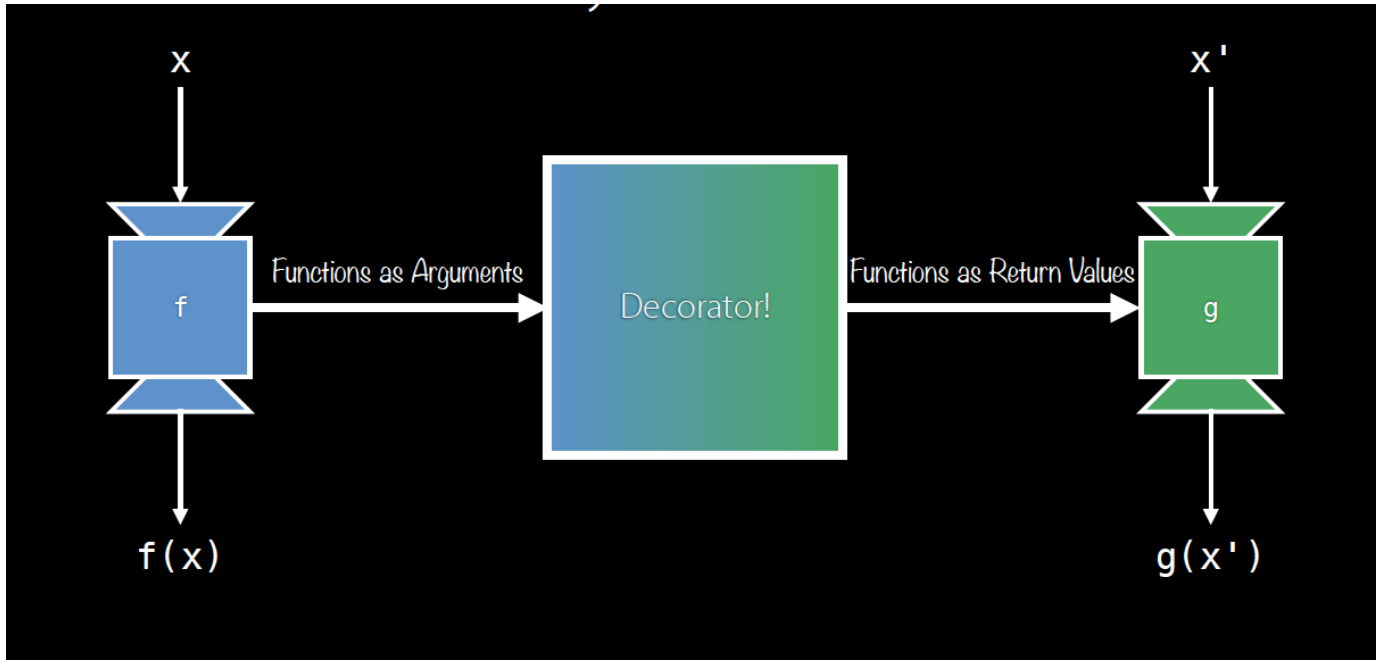
- **Reduction to a Single Value**: reduce() returns a single value, which is the final result of reducing the entire iterable.
- **Requires a Cumulative Function**: The function used must take two arguments and return a single value, allowing for data aggregation.
- **Behavior with the Initializer**: If you provide an initializer, reduce() uses it as the first argument, and the function is applied to it and the first element of the iterable. If the iterable is empty and an initializer is provided, reduce() will return the initializer itself. If the iterable is empty and no initializer is provided, it will raise a TypeError.
- **Lazy Evaluation**: Unlike map() and filter(), which return an iterator, reduce() computes the final result immediately. This means it can consume more memory when applied to large sequences.
- **Usage in the Context of Functional Programming**: reduce() is a great example of functional programming, as it allows for expressing aggregation operations clearly and concisely, avoiding explicit loops.


Decorator

# DECORATOR

- Decorators are very powerful and useful tool in Python since it allows programmers to modify the behavior of function or class.
- Decorators allow us to wrap another function in order to extend the behavior of wrapped function, without permanently modifying it.
- In Decorators, functions are taken as the argument into another function and then called inside the wrapper function.

# Decorator




- A decorator is a function that modifies the behavior of another function.
- It allows you to "decorate" a function with new functionality without modifying its source code.
- Uses the syntax @decorator_name before the function definition.

# How to Define a Decorator

```
def my_decorator(func):
    def wrapper():
        print("Something is about to happen before the function.")
        func()
        print("Something has happened after the function.")
    return wrapper
```

# Using a Decorator

```
def my_decorator(func):
    def wrapper():
        print("Something is about to happen before the function.")
        func()
        print("Something has happened after the function.")
    return wrapper
```

```
Applying the Decorator to a Function
@my_decorator
def say_hello():
    print("Hello!")

say_hello()
```

Output:
Something is about to happen before the function.
Hello!
Something has happened after the function.

# Decorators with Arguments

```
def repeat(num_times):
    # Questa funzione accetta un argomento
    def decorator_repeat(func):
        def wrapper(*args, **kwargs):
            for _ in range(num_times):
                func(*args, **kwargs)  # Chiamata alla funzione originale
        return wrapper  # Restituisce la funzione avvolgente
    return decorator_repeat  # Restituisce il decoratore

@repeat(3)  # Passiamo 3 come argomento al decoratore
def greet(name):
    print(f"Ciao, {name}!")
```

# Why Use Decorators?

1. Modularity: Separate the logic of the main function from additional functionality.
2. Reusability: Can be applied to multiple functions without repeating code.
3. Clarity: Improve code readability by making extended functionalities evident.

# Built-in Decorators

Built-in Decorators in Python

- @staticmethod: Defines a static method in a class.
- @classmethod: Defines a class method.
- @property: Allows defining access methods (getter/setter).

# @staticmethod

```
class MathUtils:
    @staticmethod
    def add(a, b):
        return a + b
print(MathUtils.add(3, 5))   # Output: 8
```

# @classmethod

```
class Person:
    def __init__(self, name):
        self.name = name

    @classmethod
    def from_string(cls, name_str):
        return cls(name_str)

p = Person.from_string("Alice")
print(p.name)  # Output: Alice
```

# @PROPERTY

```
class Circle:

    def __init__(self, radius):

        self._radius = radius    #privato per convenzione

        @property
        def radius(self):       #Ritorna il raggio del cerchio
        return self._radius

        @radius.setter
        def radius(self, value):
                """Imposta il raggio del cerchio, assicurandosi che sia positivo."""
        if value < 0:
                        raise ValueError("Il raggio deve essere positivo.")
                self._radius = value

        @property
        def area(self): #ritorna l'area del cerchio."""
                return 3.14159 * (self._radius ** 2)
```

# @PROPERTY

```
class Circle:

    def __init__(self, radius):

        self._radius = radius    #privato

        @property
        def radius(self):       #Ritorna il
        return self._radius

        @radius.setter
        def radius(self, value):
                """Imposta il raggio del cerchio, assicurandosi che sia positivo."""
        if value < 0:
                        raise ValueError("Il raggio deve essere positivo.")
                self._radius = value

        @property
        def area(self): #ritorna l'area del cerchio."""
                return 3.14159 * (self._radius ** 2)
```

```
# Utilizzo della classe
c = Circle(5)
print(c.radius)  # Output: 5
print(c.area)    # Output: 78.53975

c.radius = 10    # Modifica del raggio
print(c.area)    # Output: 314.159

# c.radius = -3
# Questo solleverebbe un'eccezione: ValueError: Il raggio deve essere positivo.
```

# Chained Decorators

Applying Multiple Decorators

- You can apply multiple decorators to a function.
- The order of decorators is important and affects the final behavior.

```
@decorator_one
@decorator_two
def my_function():
    pass
```

```
def my_function():
    pass

my_function =
decorator_one(decorator_two(my_function))
```

# Limitations of Decorators

Considerations with Decorators

- They can complicate debugging if not well documented.
- Be cautious of performance issues when applying heavy decorators to frequently called functions.

Generation of callbacks in graphical or networking libraries.

```
import tkinter as tk

def create_button_callback(message):
    """Factory function to create a button callback that prints a message."""
    def callback():
        print(message)  # Print the message when the button is clicked
    return callback  # Return the callback function

root = tk.Tk()

# Create buttons with different callbacks
button1 = tk.Button(root, text="Click Me 1", command=create_button_callback("Button 1 clicked!"))
button1.pack()

button2 = tk.Button(root, text="Click Me 2", command=create_button_callback("Button 2 clicked!"))
button2.pack()

root.mainloop()
```

# Implementation of design patterns such as the Factory Method.

```
class Circle:
    def draw(self):
        return "Drawing a Circle"

class Square:
    def draw(self):
        return "Drawing a Square"

())
```

```
def shape_factory(shape_type):
    """to create shape objects based on the shape_type."""
    if shape_type == "circle":
        return Circle()  # Return a Circle instance
    elif shape_type == "square":
        return Square()  # Return a Square instance
    else:
        raise ValueError("Unknown shape type")

# Usage of the factory function
shapes = [shape_factory("circle"), shape_factory("square")]

for shape in shapes:
    print(shape.draw
```

```
Drawing a Circle
Drawing a Square
```